

Title Page for *The American Mathematical Monthly*

When Every Parenthesis Matters

When Every Parenthesis Matters

Abstract. Fix the variable order $x_1, \dots, x_n$ and form fully parenthesized expressions using $+$, $\times$, and $/$. An inner pair of parentheses is called necessary if deleting that pair alone, and then applying the usual precedence and left-associativity conventions, changes the represented rational function. We classify the expressions in which every inner pair is necessary. A three-row local rule yields a recursive structure and the Fibonacci enumeration

$$r_2 = 3, \quad r_n = 2F_{n+1} \quad (n \geq 3).$$

Keywords: redundant parentheses, arithmetic expressions, binary trees, Fibonacci numbers, operator precedence

1. THE QUESTION Parentheses can be visibly present and mathematically idle. For example,

$$x_1 + (x_2/x_3) = x_1 + x_2/x_3,$$

whereas

$$x_1/(x_2 + x_3) \neq x_1/x_2 + x_3.$$

How many fully parenthesized expressions have no idle inner pair at all?

Fix distinct indeterminates $x_1, \dots, x_n$ in this order. A *fully parenthesized expression* is a plane binary tree whose leaves, from left to right, are $x_1, \dots, x_n$, in which every internal vertex has exactly two children and is labeled by

$$+, \quad \times, \quad /.$$

We compare expressions as elements of $\mathbb{Q}(x_1, \dots, x_n)$. For example, the fully parenthesized expression in our sense corresponding to $x_1 + x_2 \times x_3$ is $(x_1 + (x_2 \times x_3))$. The outermost pair is not a candidate pair, since deleting it changes neither the parsing nor the represented rational function. Thus the $n - 2$ candidate pairs are precisely those corresponding to edges joining two internal vertices.

Whenever a candidate pair is deleted, the resulting infix expression is parsed in the usual way: $\times$ and $/$ bind more strongly than $+$, and operations at the same precedence level associate to the left.

A pair is *necessary* if deleting that pair alone, while retaining all other pairs, changes the rational function represented by the expression. An expression is *rigid* if all its $n - 2$ candidate pairs are necessary.

For $n = 3$, the six rigid fully parenthesized expressions are

$$\begin{aligned} ((x_1 + x_2) \times x_3), & \quad ((x_1 + x_2)/x_3), & \quad (x_1 \times (x_2 + x_3)), \\ (x_1/(x_2 + x_3)), & \quad (x_1/(x_2 \times x_3)), & \quad (x_1/(x_2/x_3)). \end{aligned}$$

Thus $r_3 = 6$.

For $n = 2$, the three fully parenthesized expressions are $(x_1 + x_2)$, $(x_1 \times x_2)$, and (x_1/x_2) . None has a candidate pair, so all three are rigid vacuously and $r_2 = 3$.

Our objects are fully parenthesized binary trees, not minimally parenthesized infix strings. Fixing the tree makes the necessity of a prescribed pair an edge-local property. We restrict attention to the three operations above; adding subtraction leads to a different enumeration problem.

2. THE LOCAL RULE Call an internal child *additive* if the operation at its root is $+$. The following table determines whether the pair surrounding an internal child is necessary.

parent	left internal child	right internal child
$+$	never necessary	never necessary
$\times$	necessary exactly when additive	necessary exactly when additive
$/$	necessary exactly when additive	always necessary

Table 1. When the parentheses around an internal child are necessary.

Lemma 1 (Local rule). *Let v be a nonroot internal vertex of a fully parenthesized expression, and let p be its parent. Whether the pair surrounding v is necessary depends only on whether v is the left or right child of p and on the operation labels at p and v , as specified in Table 1.*

Proof. In the local expressions below, we suppress the outermost pair of parentheses.

Let A and B be the two children of v , and let C be the sibling subtree of v . Thus the relevant local expression is either

$$(A \circ B) \star C \quad \text{or} \quad A \star (B \circ C),$$

where $\circ$ labels v and $\star$ labels its parent.

First consider the entries declared unnecessary in Table 1. Under the stated precedence and left-associativity conventions, deleting the displayed pair gives the same rational function in each of the following cases:

$$\begin{aligned}
 (A \circ B) + C &= A \circ B + C, & A + (B \circ C) &= A + B \circ C \quad (\circ \in \{+, \times, /\}), \\
 (A \times B) \times C &= A \times B \times C, & (A/B) \times C &= A/B \times C, \\
 A \times (B \times C) &= A \times B \times C, & A \times (B/C) &= A \times B/C, \\
 (A \times B)/C &= A \times B/C, & (A/B)/C &= A/B/C.
 \end{aligned}$$

For the remaining entries, the following positive specializations show that deleting the pair changes the value. The triple in the first column records the parent operation, the side of the internal child, and the operation at that child.

case	before deletion	after deletion
$(\times, L, +)$	$(1 + 1) \times 2 = 4$	$1 + 1 \times 2 = 3$
$(\times, R, +)$	$2 \times (1 + 1) = 4$	$2 \times 1 + 1 = 3$
$(/, L, +)$	$(1 + 1)/2 = 1$	$1 + 1/2 = 3/2$
$(/, R, +)$	$1/(1 + 1) = 1/2$	$1/1 + 1 = 2$
$(/, R, \times)$	$1/(1 \times 2) = 1/2$	$1/1 \times 2 = 2$
$(/, R, /)$	$1/(1/2) = 2$	$1/1/2 = 1/2$

These numerical witnesses apply to subexpressions of arbitrary size. Indeed, for any nonempty subexpression and any prescribed positive rational value q , its variables can be assigned positive rational values so that the subexpression evaluates to q : at a $+$ -node use child values $q/2, q/2$, and at a $\times$ - or $/$ -node use $q, 1$. Since A, B, C have disjoint variable sets, their values can be prescribed independently.

It remains only to check that a local change cannot disappear higher in the tree. Put $K = \mathbb{Q}(x_1, \dots, x_n)$. Every subexpression represents a nonzero element of K , since assigning 1 to every variable gives it a positive value. At each step toward the root, the current rational function u is transformed, for some nonzero sibling function h , by one of

$$u \mapsto u + h, \quad u \mapsto hu, \quad u \mapsto u/h, \quad u \mapsto h/u.$$

Each map is injective on its domain. Hence two distinct local functions remain distinct all the way to the root. This proves the table. ■

Proposition 2 (Context independence). *Let T be a subtree of a fully parenthesized expression E , and let π be a candidate pair inside T . Then π is necessary in T if and only if it is necessary in E .*

Proof. If deleting π does not change the rational function represented by T , it does not change the rational function represented by E . Conversely, if deleting π changes the function represented by T , the injectivity argument in the proof of Lemma 1 shows that the difference persists from the parent of T to the root. Hence the function represented by E also changes. ■

3. STRUCTURE AND THE FIBONACCI NUMBERS

Proposition 3 (Structural classification). *A fully parenthesized expression with at least two leaves is rigid if and only if exactly one of the following holds.*

1. *Its root is $+$ and both children are leaves.*
2. *Its root is $\times$ and each child is either a leaf or a two-leaf sum.*
3. *Its root is $/$, its left child is either a leaf or a two-leaf sum, and its right child is either a leaf or a rigid expression.*

Here a leaf means a vertex labeled by a variable x_i , with no operation symbol, in the full binary tree representing the expression.

Proof. Suppose that the expression is rigid. If its root is $+$, the local rule prohibits every internal child, so both children are leaves. If its root is $\times$, every internal child must be additive. By Proposition 2, such a child is itself rigid, and the first case then shows that it is a two-leaf sum. If the root is $/$, the same argument applies to the left child. The local rule places no restriction on the root operation of an internal right child, but that right subtree is rigid by Proposition 2.

Conversely, consider an expression of one of the three listed forms. Every pair immediately below the root is necessary by the local rule. Every pair inside a rigid child is necessary in the whole expression by Proposition 2. Hence the whole expression is rigid. ■

Theorem 4. *Let $F_1 = F_2 = 1$. Then*

$$r_2 = 3, \quad r_n = 2F_{n+1} \quad (n \geq 3).$$

Thus

$$r_2, r_3, r_4, r_5, r_6, r_7, r_8, \dots = 3, 6, 10, 16, 26, 42, 68, \dots$$

Proof. We have already established $r_2 = 3$ and $r_3 = 6$.

For four leaves, there is one multiplication-rooted expression, with a two-leaf sum on each side. Division contributes

$$r_3 + r_2 = 6 + 3 = 9$$

expressions, according as the left block has one or two leaves. Hence $r_4 = 10$.

Now let $n \geq 5$. A $+$ -rooted rigid expression has only two leaves, and a $\times$ -rooted rigid expression has at most four leaves. Hence the root must be $/$. Its left child is either the leaf x_1 or the two-leaf sum $x_1 + x_2$, while its right child is an arbitrary rigid expression on the remaining variables. Therefore

$$r_n = r_{n-1} + r_{n-2} \quad (n \geq 5).$$

Since $r_3 = 6 = 2F_4$ and $r_4 = 10 = 2F_5$, the stated formula follows. ■

Perspective Representative work on redundant parentheses concerns syntactic grammars and unparsing under precedence and associativity [1, 2, 3]. Related enumerative work studies equivalence classes of arithmetic expressions [4, 5]. The present problem is different: the underlying fully parenthesized tree is fixed, and every prescribed inner pair must be semantically necessary.

In OEIS notation,

$$r_n = A118658(n + 2) \quad (n \geq 3).$$

Thus Theorem 4 gives a parenthesis interpretation of this sequence [6]. OEIS A155069 concerns a different model, counting minimally parenthesized infix strings formed with $+$ and $-$.

Generative AI disclosure OpenAI's ChatGPT was used for language editing and manuscript-preparation support. The author independently verified all mathematical statements, proofs, computations, and references and takes full responsibility for the content of the manuscript.

REFERENCES

- [1] Ramsey N. Unparsing expressions with prefix and postfix operators. *Software: Practice and Experience*. 1998;28:1327–1356.
- [2] Bates RM. Distinguishing redundant parentheses by syntax. *International Journal of Computer Mathematics*. 2005;82:951–967.
- [3] Lotfallah WB. Characterizing unambiguous precedence systems in expressions without superfluous parentheses. *International Journal of Computer Mathematics*. 2009;86:1–20.
- [4] Stošić I, Damnjanović I, Randelović Ž. Counting the number of inequivalent arithmetic expressions on n variables. *Filomat*. 2025;39:949–962.
- [5] Cohen B. The number of non-isomorphic arithmetic expressions that can be constructed using $+$, $-$, $\times$, and $/$. *arXiv preprint*. 2026. arXiv:2602.18522.
- [6] Sloane NJA, The OEIS Foundation Inc. The On-Line Encyclopedia of Integer Sequences, A118658 and A155069 [Internet]. [cited 2026 Jul 31]. Available from: <https://oeis.org>.